MosaicOpt: An Algebraic Foundation for Agentic Memory Databases

Anonymous Author(s)
Anonymous Institution
Anonymous City, Anonymous State, Anonymous Country
anonymous@example.com

Abstract

Production LLM agents accumulate multi-modal memory across two timescales: long-term facts and preferences, and short-term working context. A memory unit typically carries text, one or more dense vector embeddings, structured metadata, and graph relations. Useful queries against this store are conjunctive expressions that mix all four modalities — for example, "find passages about a fault-tolerance bug from the last 30 days, semantically similar to a stack trace, attributed to a known subsystem, and connected by a derivation link to a prior incident." The dominant engineering response, *multi-path recall* with application-level score fusion, has no principled semantics, no equivalence relation over plans, and breaks down on multi-vector conjunctions. We propose **MosaicOpt**, an *algebra-first* design that extends relational algebra with (i) a *materialization* operator μ that binds computed values (vector distances, BM25 scores) as first-class attributes, (ii) first-class vector, text, and graph predicates, and (iii) a graph expansion operator Γ. The optimizer is a Volcano/Cascades-style rule-based layer driven by an *equational theory* of seven identities (M1–M3, V1, J1, T1, G1), combined with a cost-based layer that explicitly accounts for approximation. We prove the identities, give counterexamples for the non-identities (N1–N3) that classical intuition suggests but approximation breaks, and sketch a prototype and evaluation plan. The argument is foundational: agentic memory is a query problem, query problems benefit from algebraic foundations, and the right foundation is an extension of relational algebra.

Keywords

agentic memory, relational algebra, query optimization, vector search, hybrid retrieval, rule-based optimizer, equational theory

1 Introduction & Motivation

Large language model (LLM) agents are no longer stateless request–response loops. Production agents — coding copilots, personal assistants, autonomous research workers, multi-step planners — accumulate state across two timescales: *long-term memory*, which persists facts, preferences, and episodic traces across sessions; and *short-term memory*, which captures the working context of an in-flight session, including tool-call history and intermediate reasoning artifacts. This split is widely accepted in the agentic-systems literature, but its consequences for the storage layer have received less attention.

The content of agentic memory is fundamentally *multi-modal*. A single "memory unit" may carry: (i) raw text (a chat snippet, a code diff, a document excerpt); (ii) one or more dense vector embeddings, often produced by different encoders for different retrieval objectives (e.g. semantic similarity, intent classification, code search); (iii) structured metadata (timestamps, source, provenance, user tags, confidence scores); and (iv) graph structure (entity relations such as `derived_from`, `cites`, `contradicts`, `member_of`). A useful query against this store is seldom a pure similarity lookup; it is a *conjunctive* expression that mixes all four modalities. "Find passages about a fault tolerance bug from the last 30 days, semantically similar to this stack trace, attributed to a known subsystem, and connected by `derived_from` to a previous incident" is representative.

The dominant engineering response is *multi-path recall*: a fan-out across specialized engines (a vector index, a full-text index, a key–value store, a graph store), each producing a ranked candidate list, followed by application-level score fusion and re-ranking. Frameworks such as Mem0, Letta, Zep, and Cognee instantiate variations of this pattern. It works, and there is no shortage of impressive product demonstrations. But the pattern has four structural weaknesses:

(1) **No principled semantics.** The semantics of a fused query is defined by the application code that combines the per-channel scores. Two implementations of "find similar memory" in the same product can have silently different meanings.
(2) **Heuristic optimization only.** Without an algebra, there is no equivalence relation over plans. The "optimizer" is a hand-tuned orchestration pipeline whose rewrites are not derived from a sound theory.
(3) **Fusion policy leaks into the substrate.** Score combination functions (e.g. weighted sum, reciprocal-rank fusion, learned rerankers) are application concerns; once baked into a query template, they are hard to change without re-engineering the data path.
(4) **Top-k per column breaks conjunctive intent.** Each vector column is typically searched independently with its own top-k, then intersected. For multi-vector memories, the intersection of per-column top-k sets is frequently empty or impoverished, because the columns are correlated, not redundant.

Our position is that these problems are not implementation details; they are symptoms of a missing foundation. **MosaicOpt** proposes an *algebra-first* design in which relational algebra is extended with a small set of operators that make multi-modal memory queries first-class, and in which the resulting algebra — not application code — is the substrate contract between query and execution. The optimizer becomes a *rule-based* layer driven by an equational theory (Section 4), with a *cost-based* layer (Section 7) choosing between physical realizations that may legitimately differ in approximation behaviour.

Figure 1 shows the resulting architecture: application policy on top, logical algebra as the substrate contract, and physical algebra as the engine layer. The HNSW bridge $H_{V,q,\tau}$ at the bottom of the

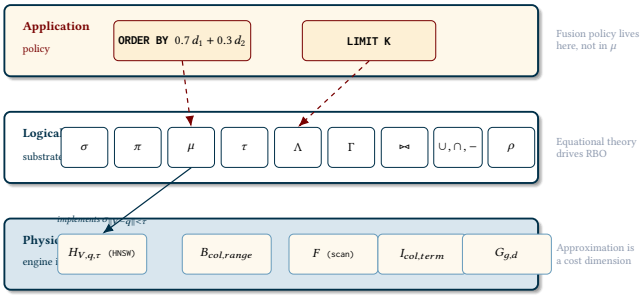


Figure 1: Three-layer architecture of MosaicOpt. The *application* layer holds the fusion policy as SQL `ORDER BY` and `LIMIT`. The *logical algebra* is the substrate contract: a small set of operators (including the new μ and Γ) over which the equational theory is stated. The *physical algebra* realizes each logical operator through concrete access paths. HNSW ($H_{V,q,\tau}$) bridges the logical vector selection $\sigma_{\|V-q\|<\tau}$ to a physical implementation whose cost depends on approximation parameters ef, M.

figure is the point at which approximation enters the system — a fact that the logical layer deliberately hides and the cost model (Section 7) makes explicit.

1.1 Why algebraic foundation?

Database systems have long benefited from formal foundations. Codd's relational algebra [1] and Selinger's cost-based optimizer [2] established that query processing benefits from a formal language for what is computable and a formal theory for what transformations preserve semantics. The Volcano/Cascades line further showed that the algebra is the *API*: a new operator is shipped with its equivalence rules and cost model, and the optimizer gains new capabilities without any central rewrite. **An algebra is not an implementation detail; it is the contract between query and execution.** When the algebra is rigorous, every query language is a syntactic surface over it, every optimizer decision is auditable, and every physical innovation is a drop-in.

Formally, an algebra provides four things that no collection of indexes and APIs can:

- **A language for expressive power** — a precise definition of what is computable in the system, and what is not.
- **An equational theory** — a body of identities that govern when one query plan is equivalent to another, the foundation on which any rule-based rewriter rests.
- **A cost-model framework** — the substrate on which cost-based optimization rests, parameterized by the same logical operators that the user wrote.
- **Composability** — new operators can be added without breaking old ones, because the algebra is the integration point. The Cascades maxim — *the algebra is the API* — makes extensibility a property of the foundation, not of the optimizer code.

Agentic memory is the *worst case* for a "bag of engines" approach. The reasons are structural:

- It spans two timescales (long-term and short-term) that demand different access patterns, lifetime semantics, and consistency guarantees.
- Its content is genuinely multi-modal: text, multiple vector columns, structured metadata, and graph relations all live on the same memory units.
- The queries that matter are *conjunctive across modalities* — a vector range intersected with a metadata filter joined with a graph reachability predicate. Ad hoc orchestration is exactly where it fails.
- There is *convergent evolution* in the field: every Agentic framework (Mem0, Letta, Zep, Cognee) has independently reinvented the same patterns (multi-path recall, score fusion, re-ranking) but with no shared vocabulary, no shared type system, and no shared notion of plan equivalence.
- The practical consequence: each framework has a different *memory API* with no formal semantics. The same SQL-like query may have silently different meanings across implementations, and porting a workload requires reading source code rather than reasoning from a specification.

1.2 Mathematical support for Agentic database development

The algebra gives database researchers and engineers a precise substrate on which to develop new query patterns and to *prove* their correctness. A new materialization pattern (e.g., a new kind of multi-modal fusion, a new join algorithm for hybrid vectors) can be expressed as a new operator with a documented equational theory — its rewrite rules are *derived* from the algebra, not guessed from benchmarks. The optimizer's plan space becomes a *derivable* structure rather than a hand-tuned mess: each new operator is added with its identities, and the optimizer gains the corresponding rewrites "for free". This is exactly the extensibility property of Cascades; the algebra is what makes it principled.

A database system built on an algebra exposes a *substrate capability*: any application that can express its query in the algebra can take advantage of the optimizer's rewrites, and any physical innovation that implements an algebraic operator improves every application that uses it. This is the foundational insight: the algebra is the *common currency* between the application and the engine. Without it, every application negotiates with every engine on its own terms; with it, an application written once is portable across engines that share the algebra, and an engine innovation is portable to every application that uses the affected operator. The algebraic foundation is therefore not a research luxury — it is the contract that makes the database ecosystem composable.

1.3 Data substrate for Agentic applications

Four representative Agentic memory use cases illustrate what applications need from a substrate:

- **Episodic recall** (debugging copilots, personal assistants, "what did I work on last week"): conjunctive vector + metadata + graph queries, where a single memory unit is retrieved only if it satisfies a similarity predicate *and* a time window *and* a graph reachability test.

- **Cross-session fact retrieval** (long-term user preferences, learned facts): single-vector or multi-vector recall with structured filters (e.g., a user-scope, a topic filter, a confidence threshold).
- **Working context assembly** (short-term memory for an in-flight session): fast filtered recall by recency + entity, where latency dominates and the top results must arrive in milliseconds.
- **Tool-trace analysis** (multi-step agents, planning): graph queries over `derived_from` and `follows` edges, joined with vector similarity on the underlying tool outputs.

All four are *query problems* — they are expressible as expressions over a memory store, and they are amenable to the same optimizer pipeline. All four benefit from a shared algebraic substrate: the optimizer's rewrites (M1, V1, J1, T1, G1) apply to each, and the cost model handles the modality-specific physics.

With MosaicOpt's algebra, an Agentic application writes its memory query in SQL (Section 5) or in the algebra directly; the substrate applies M1-style rewrites to fuse materialization with selection, applies V1/J1/T1/G1 rules to push predicates down, and applies cost-based physical-operator selection. The application does not have to know about HNSW, B-tree, or inverted indexes; the substrate hides these as physical realizations of a single logical operator. The result is that an application that today writes its own multi-path orchestrator — with hand-tuned per-channel top-k, hand-tuned fusion weights, and hand-tuned thresholds — can be rewritten as a single SQL query, and get *principled optimization for free*. This is the substrate contract in action: the algebra is what the application writes, the algebra is what the optimizer rewrites, and the algebra is what the engine realizes. The application's policy (the fusion weights, the thresholds, the top-k) stays in the SQL; the substrate's responsibility (which index, which join order, which approximation budget) is delegated to the optimizer.

1.4 Contributions

This paper makes the following contributions:

- **An extended relational algebra for agentic memory.** We add a *materialization* operator μ for binding computed values (vector distances, BM25 scores, derived attributes) so they can be referenced by `ORDER BY` and downstream predicates. We add first-class *vector*, *text*, and *graph* predicates. We add an explicit *graph expansion* operator Γ. Bag semantics and a strict logical/physical separation complete the model (Section 3).
- **An equational theory with proofs.** We state and prove a set of identities — selection/materialization commutativity (M1–M3), vector selection commutativity and join pushdown (V1, J1), the τ+$\Lambda \Leftrightarrow$ `topK` equivalence (T1), and graph expansion pushdown (G1) — and we document the *non-identities* that classical intuition suggests but approximation breaks (Section 4.5).
- **A three-layer architecture.** Application policy (SQL `ORDER BY`, `LIMIT`) is separated from the logical algebra (substrate contract) and the physical algebra (engine implementation). The substrate exposes per-column distances; fusion is the application's job.
- **A sketch of the optimizer.** A Volcano/Cascades-style RBO that uses the equational theory to generate equivalent logical plans, and a CBO that picks physical realizations under a cost model that explicitly accounts for approximation (Sections 6–7).
- **A prototype and evaluation plan.** A sketch of an open-source reference implementation, with synthetic and real-workload evaluation designed to surface the qualitative differences from multi-path recall (Sections 8–9).

The paper is intentionally foundational. We are not claiming the final word on agentic memory stores; we are claiming that the field will benefit from a shared algebraic vocabulary, just as it has in the relational world.

2 Background

2.1 Relational algebra and the classical optimizer

The relational model [1] and the relational algebra that instantiates it remain the dominant formal substrate for declarative query languages. Codd showed that a small set of operators — selection σ, projection π, Cartesian product, union, set difference, and rename — is sufficient to express any first-order query. Subsequent work refined the operator set (adding explicit $\bowtie$ for theta-join, grouping/aggregation, ordering, and limits) and crystallized in SQL.

The second foundational pillar is **query optimization**. Selinger's 1979 cost-based optimizer for System R [2] established the template that nearly every commercial RDBMS still follows: enumerate alternative orderings and access paths, estimate cost under a cardinality model, pick the cheapest. The **Volcano extensible query evaluation framework** [3] and later **Cascades** [4] generalized this to a *rule-based* outer loop driving a *cost-based* inner search, in which equivalence rules over logical operators are first-class objects, and the optimizer is itself extensible by adding rules. The insight of Cascades is that the algebra is the API: a new operator is shipped with its equivalence rules and cost model, and the optimizer gains new capabilities without any central rewrite.

Two ideas from this tradition are central to our work. First, an *equational theory* — identities over plans — is the necessary foundation for principled rewriting. Second, *logical–physical separation* is what allows physical innovations (new access methods, new join algorithms) to be plugged in without changing semantics.

2.2 Vector search and modern IR

The retrieval stack has evolved along a parallel track. Information retrieval developed the *vector space model* [5] and BM25 [6] as principled relevance scoring functions. The neural era added dense *embeddings* trained for semantic similarity [7, 8], and approximate nearest neighbour (ANN) indexes make these practical at scale. HNSW [9] is the dominant ANN structure today, providing sub-linear search with controllable recall via `ef` and `M` parameters.

A crucial property of HNSW — and of most ANN indexes — is that they are *approximate*. The result of an HNSW range query is not the exact set of points within distance τ; it is an approximation that misses some true neighbours (false negatives) and may return some non-neighbours (false positives, depending on `ef` and the distance threshold). This approximation is invisible at the level of the algebraic query language: the user wrote $\sigma_{\|V-q\|<\tau}(R)$ and presumably wants the *logical* answer. It is, however, central to

the cost model: a high-recall HNSW plan costs more than a low-recall one, and the optimizer must be able to *trade off* recall against latency.

2.3 Hybrid query systems

The last decade has seen a sustained effort to bring vector and relational together. Three lines of work are relevant.

The first is **vector-aware RDBMS extensions**, exemplified by PostgreSQL's `pgvector`, DuckDB's `vss`, and Oracle's AI Vector Search. These add a vector type, an ANN index, and vector distance predicates; integration is at the function-call level. The result is usable but architecturally shallow: there are no first-class identities for vector selection pushdown, no cost model for approximation, no principled way to rewrite a hybrid query.

The second is **purpose-built hybrid stores**: Pinecone, Weaviate, Qdrant, Milvus, Vespa. These are vector-first, with metadata filtering bolted on. Their optimization story is also pragmatic, not foundational; metadata filters are pushed through vector searches by an internal planner, but the planner's rewrites are not exposed as named identities.

The third is **multi-modal query languages**: SQL/MM, SPARQL with full-text extensions, CQL (continuous query language), and various research prototypes that mix graph, text, and vector. SPARQL in particular has a careful algebraic foundation, but its fusion with vector predicates is still an open area.

The common pattern: every system mixes the modalities by *fusing scores at the application layer*, not by *extending the algebra that defines what counts as an answer.*

2.4 The gap

What is missing is an *equational* view of multi-modal memory queries in which vector distance, BM25 relevance, and graph reachability are first-class predicates on a par with $A = c$; in which computed values (distances, scores) are first-class attributes; and in which the boundary between the application's *fusion policy* and the substrate's *evaluation strategy* is explicit and contractually enforced. That is the gap MosaicOpt addresses.

Figure 2 makes the contrast vivid. The left-hand side — multi-path recall — has no plan space, no algebraic identity, no principled rewrite. The right-hand side — the MosaicOpt approach — has all three.

3 The Extended Algebra: Syntax & Semantics

We extend the classical relational algebra with three new kinds of construct: a *materialization* operator that lifts computed values to first-class attributes, first-class multi-modal predicates (vector distance, BM25, graph reachability), and a *graph expansion* operator. Bag semantics are used throughout, matching SQL and avoiding the row-collapse anomalies of set semantics in the presence of duplicate memory units. The logical/physical separation is enforced: physical operators are introduced in Section 7 and never appear in this section.

3.1 Logical/physical separation

Following System R and Cascades, we treat the *logical* algebra as the substrate contract — what the user asks for — and the *physical* algebra as the engine contract — how it is evaluated. Equivalences hold in the logical algebra; physical operators are *implementations* of logical operators, and a logical equivalence does not in general imply that two physical realizations are interchangeable (Section 4.5, N1–N3). This separation is what makes the system *composable*: HNSW can be added, swapped, or replaced without changing the language the user sees.

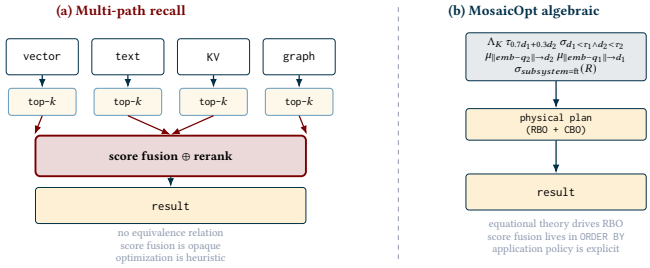


Figure 2: Multi-path recall (a) versus the MosaicOpt algebraic approach (b). In (a) four independent channels produce ranked candidate lists and application-level score fusion decides the final ranking — there is no equivalence relation over plans, so all optimization is heuristic. In (b) the user writes a single logical plan over the extended algebra; the RBO enumerates equivalent rewrites under the equational theory, and the CBO picks a physical realization. The fusion policy is explicit in the `ORDER BY` expression; the substrate has no opinion on the weights.

Operator	Syntax	Meaning
Base relation	R	The named stored relation.
Selection	$\sigma_{\varphi}(L)$	Keep rows satisfying φ.
Projection	$\pi_{A}(L)$	Keep only attributes in A.
Materialization	$\mu_{e \to c}(L)$	Per row, compute e; bind to c.
Join	$L \bowtie_{\psi} M$	Pair rows with ψ true.
Union / Inter / Diff	$L \cup M, L \cap M, L - M$	Bag set ops on compat. schemas.
Rename	$\rho_{a \to b}(L)$	Rename attribute a to b.
Ordering	$\tau_{O}(L)$	Order rows by expression O.
Limit	$\Lambda_{K}(L)$	First K rows of an ordered input.
Graph expansion	$\Gamma_{g,d}(L)$	From node-id, traverse g to depth d.

Table 1: Logical operators of the extended algebra.

3.2 Operator inventory

A *logical plan* is a term over the following operators. R denotes a stored base relation. L, M denote logical plans. φ denotes a predicate, e a materialization expression, A a set of attribute names, O an ordering expression, K a non-negative integer, c a fresh attribute name, g a graph rule (a labelled-edge pattern), and d a non-negative integer depth.

Definition 1 (Bag semantics). A relation is a multiset of tuples. All operators are multiset-preserving: σ, μ, Γ preserve row multiplicity; projection preserves multiplicity; set operations combine multiplicities additively for $\cup$, by min for $\cap$, and by subtraction (clamped at 0) for $-$. τ_{O} produces a totally ordered sequence; Λ_{K} truncates it.

Bag semantics was chosen because the substrate must be able to express "the same memory unit appears N times" (e.g. an episodic

trace where the same event is recorded with several slightly different timestamps), and because it matches the SQL semantics that application code already expects.

3.3 Predicate forms

A predicate φ is a Boolean combination of atoms over the attributes of the input plan. The atom forms are:

- **Classical.** $A = c$, $A < c$, $A > c$, $A \in S$ for constant or set c, S.
- **Vector.** $\|V_a - q\| < \tau$ where V_a is a vector-valued attribute of the input, q is a query-time bound parameter (a dense vector), and $\tau \geq 0$ is a distance threshold. Equality and other comparison operators are admissible; range ($< \tau$) is the most important form.
- **Text.** $\mathrm{BM25}(t, q) > \theta$ where t is a text attribute, q a query-time bound string, and θ a relevance threshold. We use the Okapi BM25 definition [6] with parameters (k_1, b) carried with the predicate.
- **Graph.** reachable(u, v) (boolean) and edge(u, v, ℓ) (boolean; ℓ is a label or wildcard) for nodes u, v in a graph attribute.
- **Boolean combinations.** $\varphi \wedge \psi$, $\varphi \vee \psi$, $\neg\varphi$, parentheses.

The *query-time bound parameters* (q, τ, θ) are first-class values supplied at execution time, distinct from attributes of the input. They make predicates safely *late-binding*: the algebra expression is the same across query instances, only the bound parameters change.

3.4 Materialization expressions

A materialization expression e is computed per row and bound to a fresh attribute name. Forms are:

- **Classical.** $A + B$, $f(A)$ for any scalar function f.
- **Vector.** $\|V_a - q\|$ — the distance from a vector attribute to a query-time bound vector.
- **Text.** $\mathrm{BM25}(t, q)$ — a BM25 score.
- **Boolean / arithmetic combinations** of the above.

The key property of μ is that the *result* of the computation becomes a first-class attribute of the output. Once materialized, the value is referenceable by downstream σ, by π, by `ORDER BY` expressions in τ, and by other μ expressions. This is what makes the substrate able to expose per-column distances to application-level `ORDER BY` without losing them.

Definition 2 (Fresh-name condition)**.** The new attribute name c in $\mu_{e \to c}(L)$ must not already appear in the schema of L. This is a hygiene condition, not a semantic restriction; ρ is provided to free names if needed.

3.5 Materialization: motivation and semantics

A naïve way to expose per-column distances to the application is to *inline* them in `ORDER BY` expressions: `ORDER BY` $\|V_a - q_a\| + \|V_b - q_b\|$. This is syntactically permitted by SQL extensions, but it forces the application to write the distance expression everywhere it is used, and it leaves the optimizer no room to share computation between, say, a `WHERE` filter and an `ORDER BY` rank.

μ resolves this. The pattern

$$\mu_{\|V_a - q_a\| \to d_a}\ \mu_{\|V_b - q_b\| \to d_b}\ \sigma_{d_a < \tau_a \wedge d_b < \tau_b}\ \tau_{d_a + d_b}\ \Lambda_K$$

says: "compute the distance to query a and to query b; keep rows where both are below their respective thresholds; sort by sum of distances; return the top K." The fusion policy $d_a + d_b$ is expressed once, in the τ expression. The optimizer can rewrite this plan — for example, by using M1 to commute the selection through the materializations, computing each distance only on surviving rows (Section 6).

Formally, the semantics of $\mu_{e \to c}(L)$ is the multiset $\{\, r \oplus \{c \mapsto e(r)\} \,:\, r \in L \,\}$, where $\oplus$ extends the tuple with the binding of the fresh attribute c to the value of e on r. If $e(r)$ is undefined (e.g. distance to a null vector), the row is dropped. This is a minor departure from pure relational algebra, but it is consistent with SQL's `NULL` handling and is essential for safety in the presence of missing embeddings.

3.6 Graph expansion

$\Gamma_{g,d}(L)$ takes a plan whose rows contain a node-id attribute (u, say) and a graph attribute, applies graph rule g up to depth d, and returns the *target* nodes of those traversals (joined with the originating row). The depth is inclusive of intermediate hops. g is a labelled-edge pattern: a regular expression over edge labels, evaluated against the graph. The graph itself is supplied as a relation-valued attribute; the expansion is a join between L and the traversal result.

Γ is intentionally narrow. We do not attempt to encode path queries as selection predicates; this keeps the operator set small and the rewrite theory tractable (Section 4, G1). Path information can be recovered by a μ that records the path taken.

3.7 Example queries

We illustrate the algebra on a running example. Let *Mem* be a relation of memory units with schema

$$\begin{aligned} \mathit{Mem}(&\mathit{id} : \text{int},\ \mathit{text} : \text{string},\\ &\mathit{ts} : \text{timestamp},\ \boldsymbol{\mathit{emb}} : \text{vector},\ \boldsymbol{\mathit{emb_alt}} : \text{vector},\\ &\mathit{user} : \text{string},\ \mathit{subsystem} : \text{string}). \end{aligned}$$

Let *KG* be a graph relation

$$\mathit{KG}(\mathit{src} : \text{int},\ \mathit{dst} : \text{int},\ \mathit{label} : \text{string})$$

holding derivation links between memory units. The query "find semantically similar passages in the fault-tolerance subsystem from the last 30 days, ranked by a weighted sum of two distance metrics, top 10" is

$$\begin{aligned} Q_1 &= \mathit{Mem},\\ Q_2 &= \sigma_{\mathit{subsystem}=\text{'fault-tolerance'} \wedge \mathit{ts} > \mathrm{now}() - 30d}(Q_1),\\ Q_3 &= \mu_{\|\boldsymbol{emb} - \boldsymbol{q}_1\| \to d_1}\ \mu_{\|\boldsymbol{emb_alt} - \boldsymbol{q}_2\| \to d_2}(Q_2),\\ Q_4 &= \sigma_{d_1 < \tau_1 \wedge d_2 < \tau_2}(Q_3),\\ Q_5 &= \tau_{0.7 \cdot d_1 + 0.3 \cdot d_2}(Q_4),\\ Q_6 &= \Lambda_{10}(Q_5). \end{aligned}$$

The fusion policy $0.7 \cdot d_1 + 0.3 \cdot d_2$ is application code; the substrate has no opinion on the weights. The thresholds τ_1, τ_2 are application choices too. The algebra's job is to make this expression *rewriteable* — for example, to compute d_1 and d_2 only on the post-selection rows, and to enable `topK` early termination.

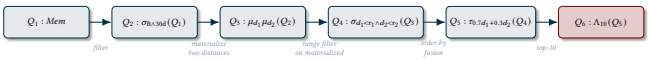


Figure 3: The running example $Q_1 \to Q_6$ as a horizontal pipeline. Each box is a logical plan; the arrows denote data flow. The fusion policy $0.7 \cdot d_1 + 0.3 \cdot d_2$ lives in the τ operator of Q_5; the substrate has no opinion on the weights. The optimizer uses M1 to push Q_4's selection below Q_3's materializations, computing d_1 and d_2 only on rows that survive the filter.

Figure 3 renders the same example as a horizontal pipeline. The visual emphasises the compositionality of the algebra: each Q_i is itself a logical plan and can be referenced as input to the next.

A graph-aware variant — "the same, but only memories linked by a `derived_from` chain to a known incident memory" — is

$$R_1 = \sigma_{id=42}(Mem) \quad \text{— the seed incident}$$
$$R_2 = \Gamma_{label='\text{derived_from}', d=2}(R_1) \quad \text{— two hops downstream}$$
$$R_3 = \sigma_{id \in \pi_{dst}(R_2)}(Q_2) \quad \text{— intersect with subsystem filter}$$

followed by the same materialization / selection / ordering / limit chain as before. The graph expansion is composable: Γ produces a relation that can be σ-, π-, and μ-ed like any other.

4 Equational Theory: Identities and Non-Identities

The optimizer (Section 6) is driven by an *equational theory*: a set of identities over logical plans. The same theory also records, explicitly, the identities that classical intuition suggests but approximation breaks — the *non-identities* — so the optimizer knows not to apply them. This section unifies both sides. We first give an overview (4.1), then the inherited classical axioms (4.2), then the new identities that the extended operators license (4.3), then the derived rewrites the optimizer uses (4.4), and finally the non-identities with counterexamples (4.5).

4.1 Overview

The equational theory of the extended algebra plays two complementary roles. *First*, it is the foundation for the rule-based optimizer (Section 6): every rewrite the RBO performs is justified by an identity — M1, V1, J1, T1, G1, or a derived combination — so the optimizer's behaviour is auditable, not opaque. *Second*, the *non*-identities (N1–N3) are the discipline that prevents the optimizer from applying rewrites that hold in classical relational algebra but break under approximation, asymmetric physical costs, or distributed semantics. The cost model (Section 7) is responsible for the physical cost of every rewrite; the equational theory is responsible for its *correctness*.

Concretely, an identity is an equation of the form `LHS = RHS` between two logical plans that hold for all instances of the free variables (relation names, attribute names, query-time bound parameters). A non-identity is a pair of plans that classical intuition might equate but the extended setting does not, with an explicit counterexample. Both are first-class objects: the RBO loads them as data, the cost model consults them when computing the "cost of identity application" term, and the executor relies on them for safety of adaptive re-planning (Section 7).

The structure of this section follows the algebra's structure: inherited axioms (the ones we use but do not re-prove) in Section 4.2; new identities (the ones the extended operators license) in Section 4.3; derived rules (combinations used directly by the optimizer) in Section 4.4; and non-identities (counterexamples that bound the optimization horizon) in Section 4.5.

4.2 Inherited foundational axioms

The following identities are standard and are assumed without re-proof. We list them because they interact with the new identities in Section 6.

- **(A1) Selection composition.** $\sigma_\varphi \sigma_\psi(L) = \sigma_{\varphi \wedge \psi}(L)$.
- **(A2) Selection distributes over union.** $\sigma_\varphi(L \cup M) = \sigma_\varphi(L) \cup \sigma_\varphi(M)$.
- **(A3) Projection idempotence.** $\pi_A \pi_B(L) = \pi_A(L)$ if $A \supseteq B$.
- **(A4) Selection–join pushdown.** If every attribute mentioned in φ is in the schema of L, then $\sigma_\varphi(L \bowtie_\psi M) = \sigma_\varphi(L) \bowtie_\psi M$.
- **(A5) Projection–join pushdown.** If A contains all attributes of L and all attributes referenced by ψ in M, then $\pi_A(L \bowtie_\psi M) = \pi_A(L) \bowtie_\psi \pi_{A \cap \text{attr}(M)}(M)$.
- **(A6) Rename is involutive on attribute names.** $\rho_{b \to a}(\rho_{a \to b}(L)) = L$.

These hold for any predicate φ in our extended form (classical, vector, text, graph), provided the attribute references are respected. We do not claim any *new* identities here; we merely note that the extended predicates do not violate them.

4.3 New foundational axioms

We introduce the seven identities that the extended algebra makes possible. Each is stated in a form that names the conditions for its applicability.

Theorem 1 (M1 — Selection commutes with materialization on the materialized column)**.** *Where c is a fresh attribute name and e is an expression depending only on attributes of L:*

$$\sigma_{c<\tau}\, \mu_{e \to c}(L) \;=\; \mu_{e \to c}\, \sigma_{e<\tau}(L).$$

Proof. Fix $r \in L$. The LHS computes $e(r)$, attaches it as c, and retains $r \oplus \{c \mapsto e(r)\}$ iff $e(r) < \tau$. The RHS evaluates $e(r) < \tau$ directly: if false, r is dropped; if true, the row is extended to $r \oplus \{c \mapsto e(r)\}$ by the μ. The two multisets are equal row-by-row. The freshness of c ensures that the predicate $c < \tau$ in LHS is unambiguous — c is defined by the μ and is not shadowed by any pre-existing attribute. ■ ■

Figure 4 visualizes the M1 rewrite. The left tree computes e on every row of R; the right tree, after M1, computes e only on rows that survive the filter. For a tight vector range, the selectivity of $e < \tau$ can be one part in 10^3 or smaller, and the saving is correspondingly dramatic.

Theorem 2 (M2 — Materialization is commutative)**.** *For fresh names c_1, c_2 and expressions e_1, e_2 depending only on L's attributes (and not on each other's outputs):*

$$\mu_{e_1 \to c_1}\, \mu_{e_2 \to c_2}(L) \;=\; \mu_{e_2 \to c_2}\, \mu_{e_1 \to c_1}(L).$$

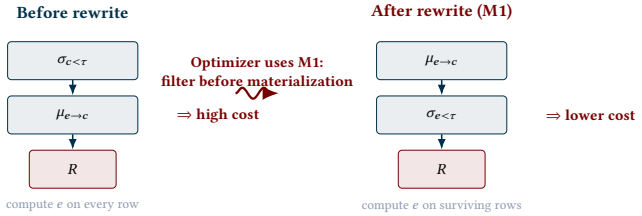


Figure 4: Visualization of the M1 rewrite. *Left*: the input logical plan materializes e on every row, then filters on $c < \tau$. *Right*: after M1, the filter is applied *before* materialization; e is computed only on rows that survive the filter. The cost saving scales with the selectivity of $e < \tau$ — substantial for tight vector ranges.

Proof. For each $r \in L$, LHS computes $e_1(r)$ and then $e_2(r)$, attaching both; RHS computes $e_2(r)$ and then $e_1(r)$. Because e_1 and e_2 depend only on r and not on each other's intermediate result, the two sequences yield the same extension $r \oplus \{c_1 \mapsto e_1(r), c_2 \mapsto e_2(r)\}$. The multisets are equal. ■ ■

Theorem 3 (M3 — Materialization through projection). *For fresh c and expression e depending only on attributes in A:*

$$\mu_{e\to c}\, \pi_A(L) \;=\; \pi_{A\cup\{c\}}\, \mu_{e\to c}(L).$$

Proof. For $r \in L$, the LHS produces $\pi_A(r) \oplus \{c \mapsto e(\pi_A(r))\}$. The RHS produces $r \oplus \{c \mapsto e(r)\}$ (using the same e since e depends only on A), then projects to $(A \cup \{c\})$. By the support of e, $e(r) = e(\pi_A(r))$, and the resulting tuples agree. The freshness of c ensures the projection is well-formed. ■ ■

Theorem 4 (V1 — Vector selections on distinct columns commute). *For distinct vector-valued attributes V_a, V_b and corresponding bound parameters and thresholds:*

$$\sigma_{\|V_a-q_a\|<\tau_a}\, \sigma_{\|V_b-q_b\|<\tau_b}(L) \;=\; \sigma_{\|V_b-q_b\|<\tau_b}\, \sigma_{\|V_a-q_a\|<\tau_a}(L).$$

Proof. The composite predicate is $(\|V_a - q_a\| < \tau_a) \wedge (\|V_b - q_b\| < \tau_b)$, which is equivalent to $(\|V_b - q_b\| < \tau_b) \wedge (\|V_a - q_a\| < \tau_a)$. By A1, the selection-composition identity, the two orderings collapse to the same composite selection. ■ ■

Remark 1. V1 is a *correctness* identity: the two plans produce the same logical result. They are not interchangeable in *cost*: the order in which the two vector selections are evaluated affects how many candidate rows each HNSW search returns, the cumulative HNSW work, and the intermediate result-set sizes. The optimizer must keep both orderings in the plan space and let the cost model choose (Section 7).

Theorem 5 (J1 — Vector selection pushdown through join). *Where V_a is an attribute of L and ψ is the join predicate:*

$$\sigma_{\|V_a-q\|<\tau}\, (L \bowtie_\psi M) \;=\; (\sigma_{\|V_a-q\|<\tau}(L)) \bowtie_\psi M.$$

Proof. Direct instantiation of A4 with the vector predicate as φ. The result is the same multiset because $\|V_a(r) - q\| < \tau$ is independent of any attribute of M. ■ ■

Theorem 6 (T1 — Limit after order equals top-K). *Where O is a total ordering expression:*

$$\tau_O\, \Lambda_K(L) \;=\; \mathrm{topK}_O(L, K).$$

Proof. By definition, $\tau_O(L)$ produces the rows of L in non-decreasing order of O, with stable tie-breaking on remaining columns; Λ_K then takes the first K. The result is precisely the K smallest rows under O. The two plans are logically equal. The interesting observation is *physical*: the LHS in the algebra may be implemented as `Sort` followed by `Limit`; the RHS as `TopKHeap` with incremental bounds. They are equivalent in result but very different in cost. ■ ■

Theorem 7 (G1 — Selection pushdown through graph expansion). *Where φ references only attributes of L (and not attributes of the graph rule's traversal result):*

$$\sigma_\varphi\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \sigma_\varphi(L).$$

Proof. Let R be the set of rows of L satisfying φ; the seed set of LHS is L, of RHS is R. Let $\mathrm{reach}(g, d, S)$ denote the set of nodes reachable from a seed set S by rule g within depth d. Monotonicity of reachability gives $\mathrm{reach}(g, d, R) \subseteq \mathrm{reach}(g, d, L)$. The σ_φ on the LHS then removes any row whose origin row is not in R (since φ depends only on the origin's attributes), so the two multisets agree. ■ ■

4.4 Derived identities

By composing the identities, additional rewrites become available. We name two that the optimizer uses directly:

- **J1+M1 (vector selection before materialization).** Combining J1 and M1 allows a join-side vector selection to be pushed below a materialization that binds a distance for sorting:

$$\begin{aligned}\sigma_{\|V-q\|<\tau}\, \mu_{\|V-q\|\to d}\, (L \bowtie M) &= \mu_{\|V-q\|\to d}\, \sigma_{\|V-q\|<\tau}\, (L \bowtie M) \quad \text{(M1)}\\ &= \mu_{\|V-q\|\to d}\, (\sigma_{\|V-q\|<\tau}(L) \bowtie M) \quad \text{(J1)}\end{aligned}$$

  The plan now computes the distance only on the surviving L rows.
- **T1 (topK replaces $\tau + \Lambda$).** Whenever the algebra has $\tau_O\, \Lambda_K$, the CBO can replace the pair with a single `topK` physical operator (Section 7).

The algebra is closed under composition: the output of any operator is a logical plan usable as input to any other. The schema rules ensure that the attributes a downstream operator references are either attributes of the input or have been introduced by a μ in the pipeline. The rewrite rules documented in Sections 4.2–4.4 — together with the standard axioms A1–A6 — therefore suffice to cover the optimization of all plans whose operators draw from the inventory of Section 3.

4.5 Non-identities and counterexamples

Some rules that look natural in a relational setting **do not hold** in the extended algebra, and a third class — physical identities — holds in the logical algebra but not in the physical algebra. We document these explicitly so the optimizer knows not to apply them. The same logical plan in two realizations can have different outputs; the optimizer must treat this as a *cost* phenomenon, not a *correctness* phenomenon.

4.5.1 N1: Approximation breaks logical equivalence.

Theorem 8 (N1 — HNSW approximation breaks logical equivalence)**.** *Logical $\sigma_{\|V-q\|<\tau}(R)$ is exactly defined. Physical $H_{V,q,\tau}(R)$ is* approximate *— it is missing some true neighbours and (depending on* `ef`*) may include some non-neighbours. A logical equivalence does not imply that two physical realizations preserve it.*

Counterexample 1. Let R consist of a single row $r = (id : 1, \boldsymbol{emb} : \boldsymbol{v})$ with $\|\boldsymbol{v} - \boldsymbol{q}\| = 0.99\tau$ (just under the threshold). An HNSW index on ***emb*** configured with a small `ef` and an unfavourable entry point may fail to reach r during the graph walk, returning an empty result. A linear scan over R returns $\{r\}$. Both are valid physical implementations of the same logical selection, and the logical identity $H_{V,q,\tau}(R) = F(R) \cap \ldots$ (whatever implementation identity) does not hold under approximation.

Remark 2. The optimizer must not assume that two implementations of a logical operator agree on output; it must model approximation as a first-class *cost dimension* with quantified recall/precision, and it must let the cost-based search explore the trade-off (Section 7).

4.5.2 N2: Similarity join asymmetry.

Theorem 9 (N2 — Similarity join asymmetry)**.** *Logically, the similarity joins $R \bowtie_{\|V_a - V_b\|<\tau} S$ and $S \bowtie_{\|V_b - V_a\|<\tau} R$ are equivalent. Physically, the two sides are not interchangeable in cost.*

Counterexample 2. Let $|R| = 10^6$ with an HNSW index on V_a, and $|S| = 10^3$ with no index. The plan "HNSW on R probed by each row of S" issues 10^3 HNSW probes, each touching a small portion of the graph. The reverse plan "HNSW on S probed by each row of R" would require either building an HNSW index on S (extra cost) or issuing 10^6 probes. The two logical plans are equivalent; the physical plans have different costs by three orders of magnitude.

Remark 3. The cost model must encode which side of a join has the index and which has the probes. The RBO can generate *both* the LHS and RHS join orderings; only the CBO can decide which is cheaper.

4.5.3 N3: Limit and order do not distribute over union.

Theorem 10 (N3 — Limit and order do not distribute over union)**.** $\Lambda_K\, \tau_O\, (R \cup S) \neq \Lambda_K\, (\tau_O(R)) \cup \Lambda_K\, (\tau_O(S))$ *in general.*

Counterexample 3. Let R contain 100 rows each with score 0, and S contain 100 rows each with score 10. Let $K = 5$ and O be the score attribute. The LHS, $\Lambda_5\, \tau_O(R \cup S)$, returns the 5 rows from R (the top 5 of the union by score). The RHS, $\Lambda_5\, \tau_O(R) \cup \Lambda_5\, \tau_O(S)$, returns the 5 rows from R (all score 0) *plus* the 5 rows from S (all score 10), giving a 10-row result. The two multisets differ.

Remark 4. The identity $\Lambda_K\, \tau_O(L \cup M) = \Lambda_K\, \tau_O(L) \cup \Lambda_K\, \tau_O(M)$, which would be convenient for distributed execution, does not hold. The optimizer must compute the union before applying the limit. This is a strong argument for keeping the *logical* plan compact and letting the *physical* planner manage the distributed execution (Section 7).

A related non-identity: $\tau_O(L \cup M) = \tau_O(L) \cup \tau_O(M)$ is formal only when the union of two ordered sequences is itself a well-defined sequence — i.e. when the user accepts that "ordered union" of two ordered bags is the interleave, not the re-sort. In practice, τ is almost always paired with Λ, and T1 already subsumes the case.

5 SQL Surface

The algebra of Section 3 is the *optimizer's* working language; it is not what application code writes. Application code writes SQL, in a small extension that is the locked-in query interface for the prototype. This section makes the two surfaces explicit, gives the desugaring from SQL into the algebra, and justifies the design decisions. The design itself is documented in `notes/decisions/2026-07-24-sql-extension.m` this section is the paper-facing treatment.

5.1 Motivation: two equivalent surfaces

The substrate contract is the logical algebra (Sections 3–4). But application code, by convention, by tooling, and by the decades of investment in SQL-based ORMs and dashboards, is expressed in SQL. The right architecture is therefore a *thin SQL surface* that desugars into the algebra, and a *round-trip* property: every SQL query expresses a plan in the algebra, and every plan in the algebra has a SQL rendering. The optimizer manipulates the algebra; the parser is the only component that sees SQL. From the application's point of view, the SQL is the *primary* surface and the algebra is an internal intermediate representation; from the optimizer's point of view, the algebra is the primary surface and SQL is a *projection* of it. The two viewpoints are not in tension because the desugaring is total, deterministic, and semantics-preserving.

The remainder of this section justifies the four SQL-surface design choices the prototype has locked in (Section 5.8), then walks through the DDL (Section 5.2), the predicate surface (Section 5.3), the materialization mechanism (Section 5.4), the graph expansion surface (Section 5.5), the formal desugaring rules (Section 5.6), and a worked example (Section 5.7).

5.2 DDL: vector type and indexes

The SQL surface adds one *type* and two *index* DDL clauses. We deliberately do not introduce a new tablespace, a new "collection" concept, or a separate vector store relation — vectors are first-class column types, and indexes are declared on the same relation that holds the data.

```
-- Vector column with optional explicit metric
ALTER TABLE memory
    ADD COLUMN emb VECTOR(768);
ALTER TABLE memory
    ADD COLUMN emb_alt VECTOR(768, METRIC := 'cosine');

-- HNSW index on a vector column
CREATE INDEX idx_emb_cos
    ON memory USING hnsw (emb vector_cosine_ops);

-- BM25 inverted index on a text column
CREATE INDEX idx_text_bm25
    ON memory USING bm25 (text);
```

The `VECTOR(N)` type is a typed array of N 32-bit floats; the optional `METRIC := '...'` clause names the distance metric (`'cosine'`,

'l2', 'ip') and is used by the CBO to *prefer* the matching HNSW index when one is available. The USING hnsw and USING bm25 index methods are the physical access paths of Section 7 exposed as DDL. Nothing in the algebra changes; the DDL is a *creation* concern, not a *query* concern.

5.3 WHERE: multi-modal predicates

The WHERE clause of a query is the primary surface for multi-modal predicates. We adopt **SQL/MM-style syntax** for all three new predicate kinds (vector, text, graph) and *keep* pgvector's distance operators for low-level use:

```
-- Vector range predicate (SQL/MM-style, first-class)
WHERE emb IS NEAR :q1 WITHIN DISTANCE :tau1
WHERE emb_alt IS NEAR :q2 WITHIN DISTANCE :tau2

-- Text relevance predicate
WHERE text MATCHES :q1 WITH SCORE > :theta

-- Graph reachability predicate
WHERE id IS REACHABLE FROM :n WITHIN 2 HOPS VIA
     'derived_from'

-- pgvector operators are still available (low-level,
     escape hatch)
WHERE emb <=> :q1 < :tau1
WHERE emb <-> :q1 < :tau1
WHERE emb <#> :q1 > :tau1
```

The design rationale for the SQL/MM style is documented and is the user's explicit preference: function-style (distance(emb, :q) < :tau) reads as a *user-defined function* — a black box the engine does not understand — and so discourages the optimizer from optimizing it. The SQL/MM form IS NEAR ... WITHIN DISTANCE reads as a *first-class* construct that the engine is committed to understanding, and gives the parser a place to anchor the desugaring rule. The same logic applies to MATCHES ... WITH SCORE and IS REACHABLE ... VIA. We retain pgvector's <=>, <->, <#> operators for two reasons: (i) backward compatibility with the existing pgvector ecosystem, and (ii) they are the most natural surface for an ORDER BY expression that needs the raw distance (Section 5.4).

5.4 Materialization via existing AS

The materialization operator μ (Section 3) is the algebra's way to bind a computed value as a first-class attribute. The SQL surface does *not* add a new LET clause; the existing AS alias is the materialization syntax. The optimizer recognizes when an AS-bound value is *reused* in WHERE and ORDER BY and triggers the M1-style rewrite (Section 4.3) automatically.

```
SELECT
    id, text, ts, subsystem,
    emb <=> :q1     AS d1,    -- materializes distance to
     :q1
    emb_alt <=> :q2 AS d2     -- materializes distance to
     :q2
FROM memory;
```

The parser treats AS d1 as binding a fresh attribute d1 to the per-row expression emb <=> :q1; the resulting logical plan has the corresponding μ operator. When the plan is later used in a WHERE d1 < :tau1 predicate or an ORDER BY 0.7 * d1 + 0.3 * d2 expression, the optimizer pushes the σ below the μ (M1) — unless the user opted into eager materialization via the MATERIALIZED hint:

```
WITH scored AS MATERIALIZED (
    SELECT id, emb <=> :q1 AS d1, emb_alt <=> :q2 AS d2
    FROM memory
)
SELECT id FROM scored WHERE d1 < :tau1;
```

The hint is a thin override; the default is for the optimizer to decide, and the equational theory of Section 4 is the formal contract that the optimizer is honouring when it pushes a σ past a μ.

5.5 Graph expansion via recursive CTE

Graph traversal is exposed as a standard SQL WITH RECURSIVE clause — no new syntax. The optimizer handles the recursion, and the resulting fixed point is fed into the rest of the plan as a normal relation.

```
WITH RECURSIVE reachable AS (
    SELECT :n AS id, 0 AS depth
    UNION
    SELECT edge.dst, reachable.depth + 1
    FROM edge JOIN reachable ON edge.src = reachable.id
    WHERE reachable.depth < 2
)
SELECT m.*
FROM memory m
JOIN reachable r ON m.id = r.id
WHERE m.subsystem = 'fault-tolerance';
```

The recursive CTE is the standard SQL construct for transitive closure; the optimizer recognizes the edge.src = reachable.id join as a Γ-style expansion and emits the corresponding Γ operator (Section 3). The WHERE reachable.depth < 2 clause is the depth bound. We do *not* introduce a new TRAVERSE keyword; the recursive CTE is a familiar idiom and avoids yet another surface construct.

5.6 Desugaring rules: SQL → algebra

The desugaring is defined as a recursive function $\mathcal{D}(\cdot)$ (read "desugar of ·") that maps SQL fragments to algebra terms. We use $\mathcal{D}\langle\cdot\rangle$ for the SQL-source flavour and $\mathcal{D}\langle\cdot\rangle_{\text{pred}}$ for the predicate flavour. The function is total on the surface syntax of Sections 5.2–5.5 and is the *only* place where the parser and the algebra meet. We state the rules informally; the implementation is the recursive-descent parser.

D1. Projection + filter + order + limit. For

```
SELECT expr AS c, ...
FROM L
WHERE phi
ORDER BY O
LIMIT K
```

the desugaring is

$$\mathcal{D}\langle \cdot \rangle \;=\; \Lambda_K\big(\tau_O\big(\sigma_\varphi\big(\mu_{e\to c}\big(\mathcal{D}\langle L\rangle\big)\big)\big)\big),$$

with μ chaining for multiple aliased expressions. `ORDER BY` is preserved exactly as written — τ_O accepts both simple `ORDER BY c` and complex `ORDER BY 0.7*d1 + 0.3*d2`, the latter referencing materialized columns.

D2. Vector predicate. $\mathcal{D}\langle$ emb IS NEAR :q WITHIN DISTANCE :t $\rangle_{\text{pred}} = \sigma_{\|emb-q\|<t}$. The bound parameter `:q` and the threshold `:t` are the query-time bound parameters of Section 3. The desugared form is a first-class algebra expression, so the optimizer's selection-pushdown rules (J1, M1) apply.

D3. Text predicate. $\mathcal{D}\langle$ text MATCHES :q WITH SCORE > :$\theta\rangle_{\text{pred}} = \sigma_{\text{BM25}(text,q)>\theta}$. The (k_1, b) parameters are carried with the BM25 index (Section 5.2) and looked up at planning time. The CBO's inverted-index bridge (Section 7) applies directly to the desugared form.

D4. Graph predicate. $\mathcal{D}\langle$ id IS REACHABLE FROM :n WITHIN k HOPS VIA $\Gamma\rangle_{\text{pred}} = \sigma_{id\,\in\,\pi_{dst}(\Gamma_{label=\Gamma',d=k}(\sigma_{id=:n}(Mem)))}$. The inner $\sigma_{id=:n}(Mem)$ is the seed, the Γ is the labelled-edge expansion of Section 3, and the outer σ is the membership test. The σ form composes with J1 and G1 (the desugared form participates in the optimizer's rewrite rules).

D5. Recursive CTE. A `WITH RECURSIVE name AS body` clause is first expanded to its *least fixed point* under standard SQL semantics (bag `UNION` with duplicate elimination, depth bound by the user's `depth < k` predicate); the resulting relation is then translated by D1–D4. Formally, with $F(R) = \mathcal{D}\langle body\rangle$ the body as a function of the recursive name's relation, the desugaring is

$$\mathcal{D}\langle \texttt{WITH RECURSIVE name AS body SELECT . . .}\rangle \;=\; \mathcal{D}\langle \texttt{SELECT . . . FROM lfp}(F)\rangle$$

The optimizer recognizes the `lfp` form and emits a Γ operator with the appropriate depth and label.

For each rule, the desugaring preserves semantics *trivially*, by construction: each rule is the definition of the corresponding SQL/MM clause in terms of the algebra's predicate and operator semantics (Section 3).

5.7 Worked example: the $Q_1 \to Q_6$ query in both forms

We illustrate the desugaring on the running example of Section 3. The full SQL surface is:

```
WITH scored AS MATERIALIZED (
    SELECT
        id, text, ts, subsystem,
        emb <=> :q1     AS d1,
        emb_alt <=> :q2 AS d2
    FROM memory
    WHERE subsystem = 'fault-tolerance'
      AND ts > now() - interval '30 days'
)
SELECT id, text, ts
FROM scored
WHERE d1 < :tau1 AND d2 < :tau2
ORDER BY 0.7 * d1 + 0.3 * d2
LIMIT 10;
```

The corresponding algebra (the Q_1–Q_6 pipeline of Section 3) is

$$\begin{aligned}
Q_1 &= Mem,\\
Q_2 &= \sigma_{subsystem=\text{'fault-tolerance'}\,\wedge\, ts>\text{now}()-30d}(Q_1),\\
Q_3 &= \mu_{\|\boldsymbol{emb}-\boldsymbol{q}_1\|\to d_1}\ \mu_{\|\boldsymbol{emb_alt}-\boldsymbol{q}_2\|\to d_2}(Q_2),\\
Q_4 &= \sigma_{d_1<\tau_1\,\wedge\, d_2<\tau_2}(Q_3),\\
Q_5 &= \tau_{0.7\cdot d_1+0.3\cdot d_2}(Q_4),\\
Q_6 &= \Lambda_{10}(Q_5).
\end{aligned}$$

The parser desugars the SQL in four steps: (1) the `WITH scored AS MATERIALIZED` is recognized as a bound sub-query and the inner `SELECT` is desugared to $\mu_{d_1,d_2}(\sigma_{subsystem,ts}(Mem))$; (2) the outer `SELECT` becomes $\pi_{id,text,ts}(\ldots)$; (3) the `WHERE` clause becomes $\sigma_{d_1<\tau_1\wedge d_2<\tau_2}$; (4) the `ORDER BY . . . LIMIT 10` is desugared to $\tau_{0.7\cdot d_1+0.3\cdot d_2}\,\Lambda_{10}$ per T1.

The optimizer then applies the rewrite rules of Section 4. The most important rewrite is M1: the predicate $d_1 < \tau_1$ is pushed below the materialization that binds d_1, so $\|\boldsymbol{emb} - \boldsymbol{q}_1\|$ is computed only on rows that pass the `subsystem = 'fault-tolerance'` filter. The user wrote the SQL in the order that reads naturally ("compute d_1 and d_2, then filter, then order"); the optimizer delivers a plan that computes d_1 and d_2 only on the survivors. The `MATERIALIZED` hint preserves the plan shape (a μ is still emitted) but the cost of that μ is reduced by the pushdown.

5.8 Why these design choices

Four choices are locked in for the prototype.

SQL/MM-style over function-style. The `IS NEAR`, `MATCHES`, and `IS REACHABLE` syntaxes are first-class clauses, not function calls. This is a deliberate signal to the optimizer (and to readers of the SQL) that the engine is committed to understanding the construct and can apply its equational theory to it. The function-style alternative `distance(emb, :q) < :tau` is permitted (and is the form pgvector users are used to) but is a *low-level escape hatch*, not the recommended surface.

Existing `AS` over a new `LET`. Materialization is expressed with the SQL `AS` alias; the optimizer recognizes when an `AS`-bound value is reused and applies M1. This avoids introducing a new clause for a concept (μ) that already has a SQL analogue (`AS`). The optional `WITH [MATERIALIZED]` hint covers the rare case where a user wants to force eager materialization.

Recursive CTE over a new `TRAVERSE` clause. Graph expansion is expressed with `WITH RECURSIVE`, which is standard SQL and a familiar idiom for transitive closure. The optimizer recognizes the form and emits a Γ operator; no new keyword is needed.

Keep pgvector's `<=>`, `<->`, `<#>` operators. The existing pgvector distance operators are preserved for backward compatibility and as a low-level surface. The SQL/MM predicates are the recommended surface, but escape hatches matter for ecosystem fit and for advanced users who want to compose raw distances in `ORDER BY` expressions.

These four choices, jointly, keep the SQL surface *thin*. The only new types are `VECTOR(N)` and the `VECTOR(N, METRIC := ...)` variant. The only new clauses are `IS NEAR ... WITHIN DISTANCE`, `MATCHES ... WITH SCORE`, `IS REACHABLE ... VIA`, and the optional `[MATERIALIZED]` hint. Everything else — `SELECT`, `FROM`,

`WHERE`, `AS`, `ORDER BY`, `LIMIT`, `WITH RECURSIVE`, `JOIN`, `GROUP BY`, aggregates — is standard SQL. The design principle is the *minimum new surface* that still lets the algebra be the substrate contract.

6 From Algebra to Optimizer: RBO

The rule-based optimizer (RBO) is the first phase of the optimizer pipeline. It operates on **logical plans** and applies the identities of Section 4 to enumerate equivalent plans. The output is a (typically modest) set of logically equivalent plans that the CBO will then consider for realization.

6.1 RBO role

The RBO's job is **not** to pick a plan. Its job is to generate a *plan space* in which the CBO can search effectively. Concretely, the RBO uses the equational theory to:

(1) **Apply selection pushdown** (A4, J1, G1) so that selective filters are evaluated early and reduce the input to expensive downstream operators.
(2) **Commute vector and classical selections** (V1, A1) so that the more selective predicate runs first.
(3) **Reorder materializations** (M2) so that independent materializations can be evaluated in any order — this matters when the physical realizations have different costs (one HNSW scan, one BM25 scan, for example).
(4) **Expose topK opportunities** (T1) by collapsing $\tau_O\,\Lambda_K$ into a single `topK` operator that the physical planner can implement with a bounded heap.

The RBO does *not* decide which access path (HNSW vs. linear scan, B-tree vs. heap) to use. It only reshapes the logical plan.

6.2 Example transformations

Returning to the Q_1–Q_6 example of Section 3, the input logical plan is

$$\Lambda_{10}\ \tau_{0.7\cdot d_1+0.3\cdot d_2}\ \sigma_{d_1<\tau_1\wedge d_2<\tau_2}$$
$$\mu_{\|\boldsymbol{emb_alt}-\boldsymbol{q}_2\|\to d_2}\ \mu_{\|\boldsymbol{emb}-\boldsymbol{q}_1\|\to d_1}\ \sigma_{subsystem=\text{'ft'}\wedge ts>\text{now}()-30d}(Mem).$$

The RBO applies the following rewrites in order:

(1) **Selection composition** (A1): $\sigma_{d_1<\tau_1\wedge d_2<\tau_2}$ becomes the composition of two σs, one per predicate.
(2) **Selection–materialization commutativity** (M1), per predicate: each $\sigma_{d_1<\tau_1}$ is rewritten to $\mu_{\ldots\to d_1}\,\sigma_{\|\boldsymbol{emb}-\boldsymbol{q}_1\|<\tau_1}$.
(3) **Selection–selection pushdown** (A1 again, plus V1 for the vector part): the two σs commute, and the one with the lower expected selectivity runs first (a heuristic we encode as a rule with a cost precondition).
(4) **Classical selection pushdown** (A4): the classical σ on *subsystem* and *ts* is pushed to the base scan.
(5) **topK fusion** (T1): the trailing $\tau_{0.7\cdot d_1+0.3\cdot d_2}\,\Lambda_{10}$ is recognized as a `topK`.
(6) **Materialization commutation** (M2): the two independent μs can be evaluated in either order; the RBO records both plans.

The output is a small set of logical plans, all guaranteed to be semantically equivalent, but each with a distinct shape (filter order, materialization order, topK vs. sort+limit). The CBO then chooses among them.

Physical	Implements	Notes
$H_{V,q,\tau}$	$\sigma_{\|V-q\|<\tau}$	HNSW range scan; params `ef`, M.
$B_{col,r}$	$\sigma_{col\in r}$	B-tree range scan.
$F(R)$	base scan	Sequential file scan.
$I_{t,q,\theta}$	$\sigma_{\text{BM25}>\theta}$	Inverted index probe.
$G_{g,d,dir}$	$\Gamma_{g,d}$	Graph traversal; *dir* is direction.
Sort(O)	τ_O	Full sort.
TopK(O,K)	$\tau_O\,\Lambda_K$ (T1)	Bounded-heap top-K.
Materialize$(e\to c)$	$\mu_{e\to c}$	Computes e and binds c.
HashJoin, NLJ, MergeJoin	$\bowtie_\psi$	Standard join algorithms.
Partition, HashBuild, HashProbe	hash-based operators	Standard enforcers.

Table 2: Physical operators and the logical operators they implement.

6.3 Plan space pruning

Without the RBO, the CBO would have to consider every shape of every plan: for n joins, the search space is $O(n!)$; for k materializations, the space is $O(k!)$ per join ordering; and so on. The RBO prunes this space in two ways:

- **Equivalent classes.** The RBO groups plans into equivalence classes under the rewriting rules. The CBO need not consider plans within a class that are guaranteed by the RBO to be suboptimal (e.g. plans where a non-selective filter runs before a selective one).
- **Canonical form.** Each equivalence class has a chosen *canonical form* (e.g. selections pushed as low as possible, materializations clustered, topK fused). The CBO enumerates realizations of the canonical form.

The effect is that the CBO's search is over *plan shapes* and *physical realizations* jointly, but the *number of shapes* is small (typically 1–3 for the agentic-memory queries we have explored).

6.4 Style: Volcano/Cascades

Our RBO is modeled on the Cascades framework [4]. Equivalence rules are first-class objects, organized by operator pair (e.g. one rule for $\sigma\times\mu$, another for $\Gamma\times\sigma$). A memo structure caches explored expressions and their cost bounds. The implementation language is intentionally simple (Section 8); the *interesting* code is the rules, not the engine.

The key consequence: *adding a new operator is adding rules.* There is no central optimizer loop to modify. A contributor who wants to support, say, a temporal range join, writes the operator, the equivalence rules that reference it, and the cost-model entries — and the optimizer gains the new capability. This is the property that makes the algebra the API.

7 Cost Model & CBO

The cost-based optimizer (CBO) is the second phase. It takes the equivalence classes produced by the RBO and selects a *physical plan* — a tree of physical operators — under a cost model.

7.1 Physical operator inventory

The physical operators are the *implementations* of the logical operators. Each physical operator carries parameters that the CBO can tune.

7.2 Cost dimensions

The cost model has four dimensions. The first three are classical; the fourth is new.

- **I/O cost.** Page reads and writes. Dominant for cold data; mitigated by indexes.
- **CPU cost.** Distance computations, BM25 scoring, tuple materialization. For vector queries, the per-probe distance computation is the dominant component.
- **Memory cost.** Working set size, in particular the size of intermediate hash tables and the priority queue for `TopK`.
- **Approximation cost.** The *quality penalty* of choosing an approximate physical operator over an exact one. For HNSW, this is expressed as $1 - \text{recall}(ef, M)$; for the `topK` physical operator, this is 0 (it is exact, given a correct scoring function). The CBO expresses approximation as a constraint: "the chosen plan must have recall $\geq \rho_{\min}$", or as a term in the cost function weighted by application preference.

The cost model is *additive across operators in a pipeline*, weighted by estimated cardinalities at each boundary. The estimation primitives (cardinality, distinct-value count, range selectivity) are themselves non-trivial in the multi-modal setting; we discuss their realization in the prototype (Section 8).

7.3 Logical-to-physical bridge

For each logical operator, the CBO enumerates a set of candidate physical realizations. The bridge is a set of *implementation rules* in the Cascades sense:

- $\sigma_{\|V-q\|<\tau}(R)$ may be realized as $H_{V,q,\tau}(R)$ if an HNSW index exists on V; otherwise as $\mu_{\|V-q\|\to d}\, F(R)$ followed by $\sigma_{d<\tau}$. The CBO also considers the *hybrid* plan: HNSW with a small `ef` for candidates, followed by an exact re-evaluation of the threshold.
- $\sigma_{\text{BM25}>\theta}(R)$ may be realized as $I_{t,q,\theta}(R)$ if an inverted index exists on t; otherwise as a full scan with BM25 scoring.
- $\tau_O\, \Lambda_K(R)$ is realized as $\text{TopK}(O, K)(R)$, per T1. The CBO may also consider $\text{Sort}(O)(R)$ followed by $\text{Limit}(K)$, useful when the cost of sorting is amortized over a downstream group-by.

7.4 Adaptive execution

Static cost estimates are notoriously wrong, especially for vector workloads where selectivity is hard to predict from small samples. The prototype is designed to support **adaptive execution**: at well-defined plan points (e.g. after an HNSW scan returns its candidate set), the executor re-consults the cost model with observed cardinalities and may switch to a different sub-plan. The algebra is unchanged; only the physical subtree is replaced. This is a feature enabled by the logical/physical separation: an adaptive switch is *safe* by construction, because the two physical subtrees are both implementations of the same logical operator.

8 Prototype Implementation (Sketch)

The prototype is a reference implementation of the extended algebra and its optimizer. The goal is not production performance; it is a *shared artifact* the research community can read, run, and extend.

8.1 Language and framework

We propose a Python core with a thin C++/Rust acceleration layer for hot loops. Python keeps the algebra and optimizer code readable and inspectable; the native layer handles HNSW probes, BM25 scoring, and large-scale materialization. The vector index is `hnswlib`; the inverted index is a Tantivy-style posting list; the graph is a CSR or adjacency-map representation depending on scale.

8.2 Module structure

- `algebra/`. The logical operators, their constructors, the predicate language, and the rewriting rules (Section 4) as data.
- `physical/`. The physical operators, including index wrappers.
- `optimizer/`. The Cascades-style RBO and the cost-based search.
- `cost/`. The cost model: cardinality, distinct-value, and range-selectivity estimators, plus I/O/CPU/memory/approximation cost functions and the *statistical summaries* that feed them.
- `runtime/`. The executor: pipeline construction, adaptive re-planning hooks, and the push/pull dataflow.
- `frontend/`. A SQL-like surface that lowers to the algebra, with the fusion policy expressed in `ORDER BY` and `LIMIT` clauses.
- `benchmarks/`. Synthetic and real-world workloads (Section 9).

8.3 Open-source considerations

The prototype will be released under a permissive license (Apache 2.0). The initial release targets read-only queries against a single-node, single-user data set, matching the design-brief scope. We deliberately defer the write path, multi-tenancy, and distributed execution to follow-up work (Section 10). The point of the first release is to *make the algebra real*: a working system that the community can stress-test, profile, and argue with.

8.4 Status

The prototype is at the design stage. The algebra module and the rule set are the most mature components; the cost model and adaptive executor require empirical calibration, which depends on having a working system to measure. Section 9 describes the evaluation plan that closes this loop.

9 Evaluation (Plan)

The evaluation is designed to answer three questions: (Q1) Does the algebra correctly express the multi-modal queries we care about? (Q2) Does the RBO generate plans that outperform a hand-tuned multi-path orchestrator on realistic agentic workloads? (Q3) Does the cost-based choice of physical realization (trade-off between HNSW, linear scan, and exact re-evaluation) match the application's recall budget?

9.1 Workloads

We will run two workload families.

- **Synthetic.** A configurable generator that produces memory units with controlled distribution of embeddings (Gaussian mixture on a unit hypersphere), metadata (zipfian), and graph structure (preferential attachment). This lets us sweep parameters (vector dimensionality, number of vector columns, selectivity

of metadata filters, graph density) and measure the optimizer's behaviour on a known ground truth.
- **Real-world agentic traces.** A corpus of memory traces from open-source agent benchmarks (e.g. SWE-bench, HotpotQA-derived agent logs, coding-assistant sessions) reformulated as memory stores. Recall is measured against a held-out query log.

9.2 Metrics

- **Latency.** End-to-end query time, p50 / p95 / p99. The dominant signal for agentic applications where the user is waiting.
- **Recall@K.** The fraction of ground-truth top-K rows that the optimizer's plan returns, under varying HNSW `ef` settings.
- **Plan count.** The number of distinct logical plans in the equivalence class of a query. A smaller number is better (it means the RBO has converged), but a non-zero number is required (it means the cost model has something to choose).
- **Plan quality.** The cost ratio between the optimizer's chosen plan and a brute-force enumeration of all physical realizations of all logical plans (where feasible).

9.3 Baseline

The baseline is a **multi-path orchestrator** modelled on Mem0 / Letta / Zep: parallel recall across vector, BM25, and graph engines, with application-level score fusion. The orchestrator is hand-tuned (its parameters are chosen to be reasonable defaults). The hypothesis is that MosaicOpt matches it on simple queries and substantially outperforms it on queries that mix multiple vector columns with conjunctive filters, because the algebra expresses the conjunction directly and the optimizer can push selections through materializations (J1, M1).

9.4 Out of scope for the current phase

The current evaluation is single-node, single-user, read-only. Write performance, multi-tenancy, distributed execution, and production-grade durability are deferred to follow-up work. We are explicit about this scope because the contributions of this paper are foundational, not systems-engineering.

10 Discussion & Future Work

MosaicOpt is a foundation, not a finished product. We close by naming the limitations of the current proposal, the open problems that follow from them, and the broader research context in which the work sits.

10.1 Limitations

- **Single-node, single-user.** The algebra does not yet model partitioning, replication, or isolation. The cost model has no term for cross-node I/O. A distributed extension is sketched below.
- **Read-only.** There is no write path. The algebra is defined on a snapshot of the memory store. A write algebra would need to model incremental updates to materialized views and to the cost model's statistical summaries.
- **Approximation is a cost, not a contract.** The cost model treats recall as a *dimension*; the algebra treats queries as *exact*. There is a real tension here, and we have not yet resolved whether the *user-facing* language should expose approximation as a hint (`hint approx_recall=0.9`) or as a contract (`requires recall ≥ 0.9`).
- **Cost model calibration.** The cost model requires empirical calibration that we have not yet performed. A poorly calibrated cost model is worse than no cost model, because it gives the CBO a false confidence.

10.2 Open problems

- **Write path and incremental maintenance.** Materializations (μ) and indexes (HNSW, inverted) need to be kept consistent with the base relations. For a memory store with append-mostly workloads, a log-structured write path with deferred index updates is the natural design.
- **Multi-tenancy.** The algebra does not currently carry a tenant identity through plans. Adding `tenant` as a first-class attribute and the corresponding selection-pushdown rules is a straightforward extension, but the cost model must learn to factor tenant cardinality.
- **Distributed execution.** The non-identity N3 (limit and order do not distribute over union) is a particular obstacle for distributed `topK`. A natural research direction is a *partition-aware* topK algorithm that respects N3 and admits a physical operator with bounded approximation.
- **Learned indexes.** Recent work on learned indexes [10] suggests that index structures can be replaced by learned models. The logical/physical separation makes this a drop-in change: a learned index is a physical realization of a selection operator. The algebra and rewrite rules are unchanged.

10.3 Connection to other research

The algebra draws on three threads.

- **Linear algebra.** Vector distance is $\|V - q\|$; an HNSW probe is a linear-algebra computation. There is a natural research direction in *fusing* linear-algebra kernels with relational operators at the physical level — for example, a join that, in the same pipeline, computes the distance to a query vector.
- **Probabilistic databases.** Approximation in HNSW is, formally, a probability distribution over possible outputs. The cost model's "approximation dimension" is a step toward treating the database as a probabilistic system; a full treatment in the style of *probabilistic databases* [11] is a natural extension.
- **Programming-language theory.** The materialization operator μ is, in spirit, a `let` binding in a query language. There is a natural correspondence with the *let-binding* extensions of relational algebra used in query-language research, and the freshness condition for c is a hygienic-rename condition. A type-theoretic treatment of the algebra would strengthen the foundation and make the rewrite rules easier to state and prove.

10.4 Closing

The argument of this paper is simple: agentic memory is multi-modal, multi-modal is a query problem, query problems benefit from foundations, and the right foundation is an extension of relational algebra. The algebra is small, the rewrite rules are provable, and the optimizer is a direct consequence. We do not claim to have solved agentic memory; we claim that the field will benefit from a

shared vocabulary, and we offer MosaicOpt as a starting point for that conversation.

Acknowledgments

This draft builds on the project design brief (`docs/brief.md`). Sections 3–4 derive from the brief's algebra-design decisions. The authors thank the open-source communities of HNSW (`hnswlib`), Tantivy, DuckDB, and Cascades-style optimizers for the building blocks that make the prototype tractable.

References

[1] E. F. Codd. A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6):377–387, 1970.

[2] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pages 23–34, 1979.

[3] G. Graefe. Volcano—An Extensible and Parallel Query Evaluation System. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.

[4] G. Graefe. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, 1995.

[5] G. Salton, A. Wong, and C. S. Yang. A Vector Space Model for Automatic Indexing. *Communications of the ACM*, 18(11):613–620, 1975.

[6] S. Robertson, S. Walker, and S. Jones. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, NIST Special Publication 500-225, pages 109–126, 1995.

[7] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient Estimation of Word Representations in Vector Space. In *Proceedings of the ICLR Workshop*, 2013.

[8] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP*, pages 3982–3992, 2019.

[9] Y. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.

[10] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. The Case for Learned Index Structures. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*, pages 489–504, 2018.

[11] N. Dalvi and D. Suciu. Efficient Query Evaluation on Probabilistic Databases. *The VLDB Journal*, 16(4):523–544, 2007.

[12] P. Chhikara, D. Khare, R. Mundra, et al. Mem0: Towards Production-grade Long-term Memory in LLM Agents. *arXiv preprint arXiv:2504.19413*, 2025.

[13] S. Packer, S. Wooders, K. Lin, et al. Letta: Towards a Living Software Platform for Memory-augmented Language Model Agents. *arXiv preprint arXiv:2410.06130*, 2024.